JavaScript Dasar 2

SpankCode

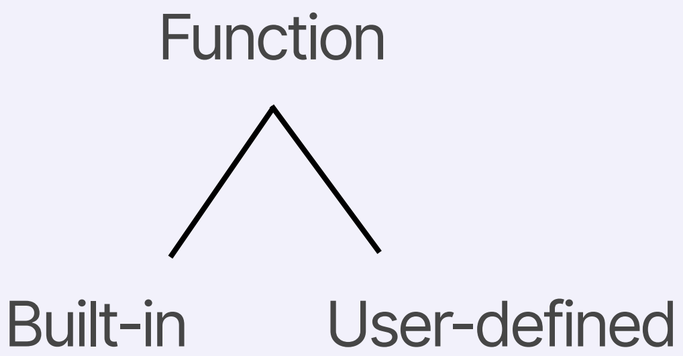
@luckyabdillah

Spank your bugs away

Function

Function merupakan **blok kode** yang bisa **dipakai ulang** untuk melakukan tugas tertentu.

DRY (Do-not Repeat Yourself)


Function
Built-in
User-defined

Built-in Function

Sudah disediakan, tinggal “panggil” saja

- alert(), prompt(), confirm().
- String: charAt(), slict(), split() toLowserCase(), toUpperCase().
- Math: sin(), cos(), tan(), random(), round(), floor(), ceil().
- dan masih banyak yang lainnya.

User-defined Function

Function yang dibuat sendiri

```
function sayHello(namaAwal, namaAkhir) {
  return “Hello ” + namaAwal + “ “ + namaAkhir
}
```

Parameter

Variabel yang dimiliki function, digunakan untuk menampung nilai saat function dipanggil. Parameter biasanya digunakan di dalam blok kode

Argument

Nilai yang dikirimkan ke parameter saat function dipanggil.

Return Value

Sebuah function bisa memiliki nilai kembalian (return) dan juga tidak.

Menjumlahkan dua volume kubus

```
function jumlahkanDuaVolumeKubus(a, b) {
  //
}
```

Refactoring

Sebuah proses mengubah kode menjadi “lebih baik”
tanpa mengubah fungsionalitasnya.

Variable Scope

Function scope vs. Block scope

Function Scope

Variabel yang dideklarasikan hanya hidup di
dalam fungsi tempat ia dibuat.

Tidak peduli berapa banyak blok { ... } di dalam fungsi, variabel
tetap bisa diakses selama masih dalam fungsi tersebut.

Function Scope

Contoh klasik: JavaScript **versi lama** dengan var.

```
function test() {
    if (true) {
        var x = 10;
    }
    console.log(x); // 10 → masih bisa diakses karena function scope
}
```

var di JavaScript tidak peduli blok, selama dalam **fungsi yang sama**, variabelnya tetap **ada**.

Block Scope

Variabel hanya hidup di dalam blok { ... } tempat
ia dideklarasikan.

Begitu keluar dari blok, variabel tidak lagi bisa diakses.

Block Scope

Contoh: C, C++, Java, dan juga JavaScript (dengan **let** atau **const**).

```
function test() {
    if (true) {
        let y = 20;
        console.log(y); // 20
    }
    // console.log(y); // ERROR → y hanya hidup di dalam blok if
}
```

var

let

const

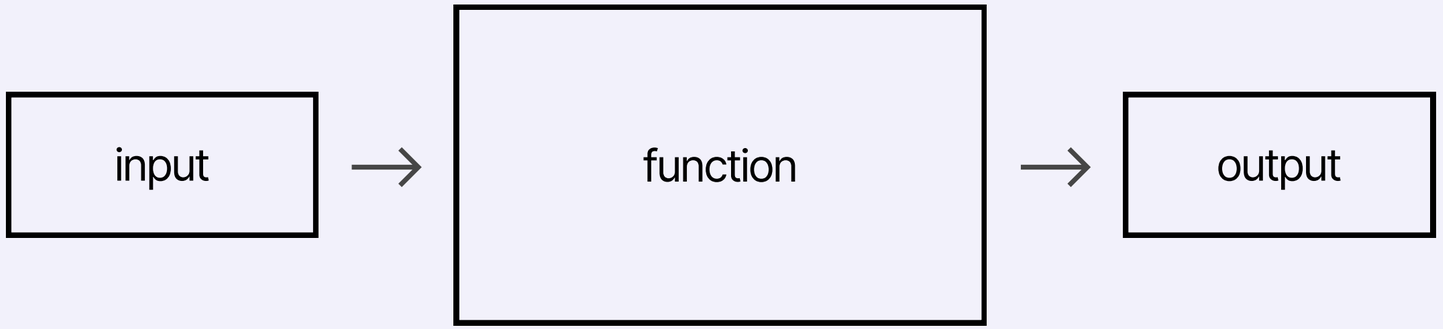
Perbedaan

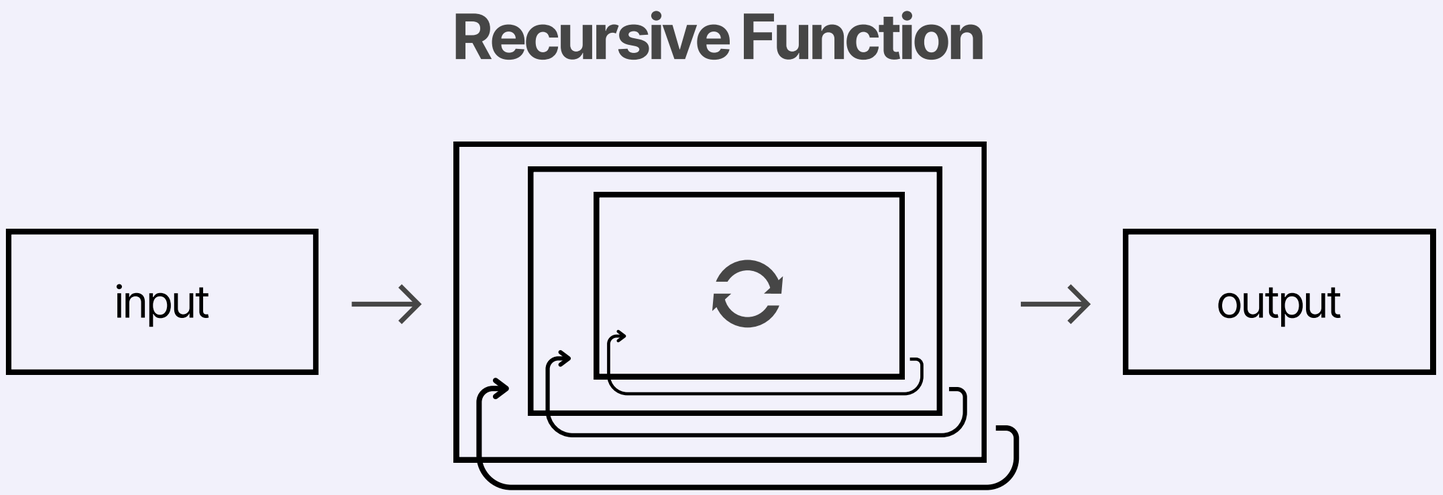
#	var	let	const
Scope	function	block	block
Re-declaration	bisa	tidak	tidak
Mutability	bisa	bisa	tidak
Global object	ya	tidak	tidak

Rekursif

Sebuah fungsi yang **memanggil dirinya** sendiri.

Normal Function

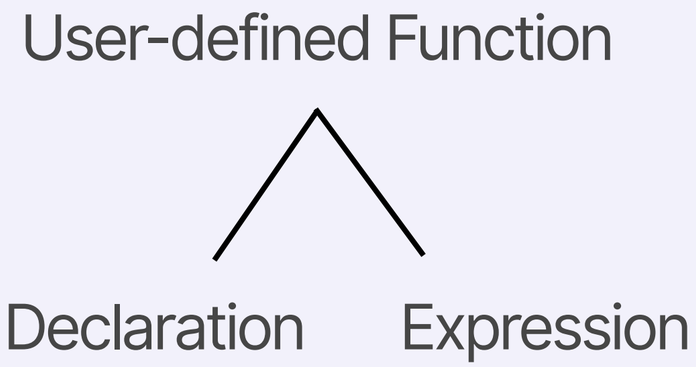



Recursive Function
input
output

```
function tes() {
  return tes()
}

function tampilAngka(n) {
  console.log(n)
  return tampilAngka(n-1)
}
```

```
function factorial(n) {
  if (n ==0) return 1
  return n * factorial(n-1)
}
```


User-defined Function
Declaration
Expression

Declaration

```
function jumlahDuaBilangan(a, b) {
  const total = a + b
  return total
}
```

Expression

```
const jumlahDuaBilangan = function(a, b) {
  const total = a + b
  return total
}
```

Perbedaan

Declaration	Expression
Dapat ditulis dimanapun (hoisting)	Linear (harus didefenisikan dahulu)
Mudah dipahami	Lebih powerful: Closure Parameter untuk function lain IIFE

Pilih yang mana?

Declaration vs. Expression

Array

Tipe data yang digunakan untuk mendeskripsikan **kumpulan elemen** (nilai/variabel), yang tiap-tiap elemen nya **memiliki index**.

```
const mhs1 = “Jainal”
const mhs2 = “Nabil”
const mhs3 = “Pandu”
.....
const mhs2935 = “Kharisman”
```

```
const hari = [“Senin”, “Selasa”, “Rabu”, “Kamis”, “Jumat”]
const mahasiswa = [“Jainal”, “Nabil”, “Pandu”, “Kharisman”]
```

- Kumpulan pasangan dengan **key** dan **value**.
- Key adalah index pada array yang **dimulai dari 0**.
- Array memiliki fungsi **length** untuk menghitung jumlah elemen di dalamnya.
- Elemen pada array **boleh** memiliki **tipe data** yang **berbeda**.

```
const mahasiswa = [“Jainal”, “Nabil”, “Pandu”, “Kharisman”]
console.log(mahasiswa[0])
console.log(mahasiswa.length())
```

```
function sayHello() {
  return “Hello, world!”
}

const angka = [21, -255, 3.14, 914]

const myArr = [“Jainal”, 2, true, sayHello, [1, 2, 3]]
```

```
// menambah isi array

let arr = []
arr[0] = “Jainal”
arr[1] = true
arr[3] = “Ganteng”
```

// menghapus isi array

let arr = [“Jainal”, “Pandu”, “Nabil”, “Risman”]
arr[1] = undefined

```
// array method

let arr = [“Jainal”, “Pandu”, “Nabil”, “Risman”]

console.log(arr.length())
console.log(arr.join())
```

```
// array method – push & pop

let arr = [“Jainal”, “Pandu”, “Nabil”, “Risman”]

arr.push(“Tiwi”, “Kezya”, “Shanessa”) // menambah elemen ke akhir
arr.pop() // menghapus elemen terakhir
```

```
// array method – unshift & shift

let arr = [“Jainal”, “Pandu”, “Nabil”, “Risman”]

arr.unshift(“Tiwi”, “Kezya”, “Shanessa”) // menambah elemen ke awal
arr.shift() // menghapus elemen awal
```

```
// array method – splice

let arr = [“Jainal”, “Pandu”, “Nabil”, “Risman”]
// splice(indexAwal, mauDiHapusBerapa, elemenBaru1, elemenBaru2)

arr.splice(2, 0, “Shanessa”, “Kharisman”)
```

```
// array method – slice

let arr = [“Jainal”, “Pandu”, “Nabil”, “Risman”]
// splice(awal, akhir)

let arr2 = arr.splice(1, 3)
```

```
// array method – foreach

let arr = [“Jainal”, “Pandu”, “Nabil”, “Risman”]
// item.forEach(callback)

arr.forEach(function (element, index) {
  console.log(e)
})
```

```
// array method – map

let arr = [“Jainal”, “Pandu”, “Nabil”, “Risman”]
// item.forEach(callback)

let arr2 = arr.map(function (element) {
  return element + “ ganteng”
})
```

```
// array method – sort

let arr = [1, 2, 6, 8, 7, 3, 5, 4, 20, 10]


arr.sort()
arr.sort(function (a, b) { // pakai compare function
  return a - b
})
```

```
// array method – filter

let arr = [1, 2, 6, 8, 7, 3, 5, 4, 20, 10]

let arr2 = arr.filter(function (e) {
  return e > 3
})
```

```
// array method – find

let arr = [1, 2, 6, 8, 7, 3, 5, 4, 20, 10]

let arr2 = arr.find(function (e) { // tidak mengembalikan array
  return e > 3
})
```

```
// array method – concat

let arr = [“Jainal”, “Pandu”, “Nabil”]
let arr2 = [“Risman”, “Shanessa”]

arr.concat(arr2, [“Tiwi”, “Kezya”])
```

```
// array method – reduce

let arr = [1, 2, 6, 8]
// reduce(callback, initialValue)

let sum = arr.reduce(function (acc, cur) {
  return acc + cur
}, 0)
```

// array method – reduce

acc (accumulator) → hasil sementara yang dibawa ke iterasi berikutnya.
curr (currentValue) → elemen array yang sedang diproses saat ini.

Step by step:

1. acc = 0, curr = 1 → return 1
2. acc = 1, curr = 2 → return 3
3. acc = 3, curr = 6 → return 9
4. acc = 9, curr = 8 → return 17

```
// array method – some & every

let arr = [1, 2, 6, 8]

arr.some(function (el) {
  return el > 5
}) // true

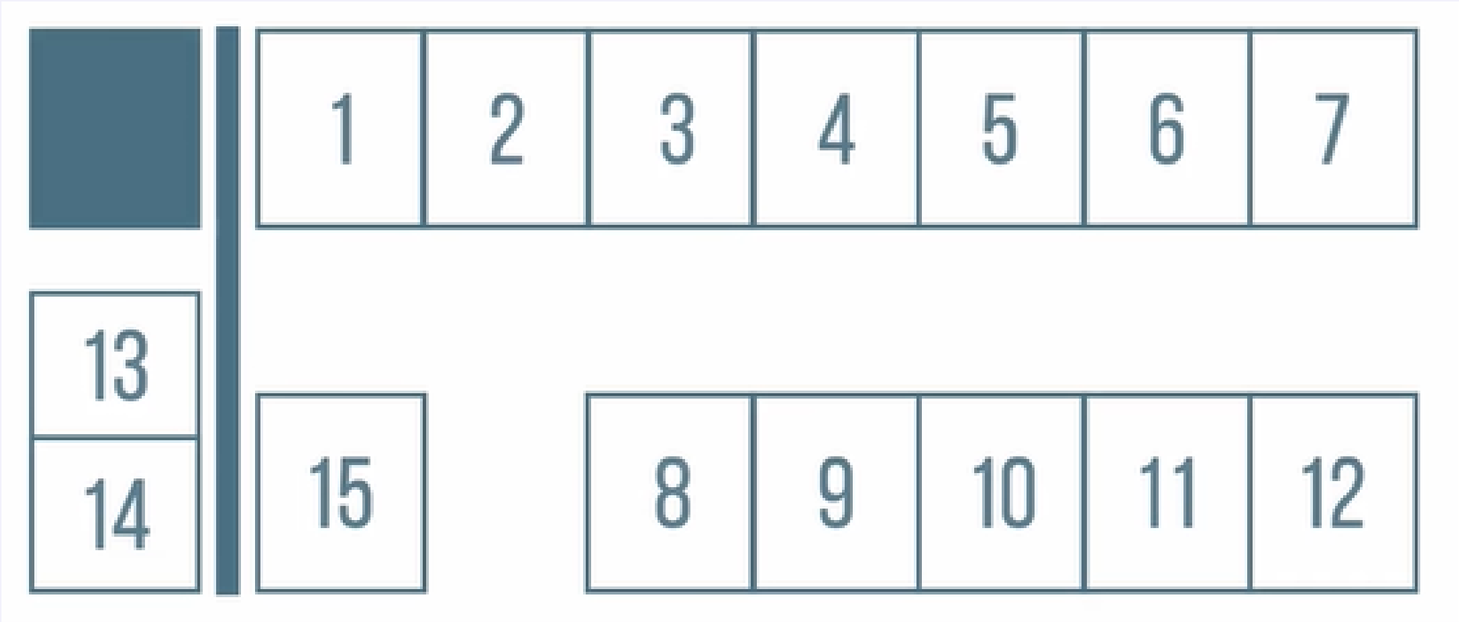
arr.every(function (el) {
 return el > 5
}) // false
```



Quiz Time

Gatau wak mau nulis apa, lorem ipsum aja ya

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer elementum metus nec viverra suscipit. In feugiat magna sapien, varius pellentesque dolor fringilla non. Mauris vitae lacinia leo, vitae eleifend lorem. Quisque facilisis nulla erat, at commodo lacus ultrices nec. Nam at orci in augue auctor rutrum sed id massa.


1
2
3
4
5
6
7
13
14
15
8
9
10
11
12

Pengelolaan Penumpang

Penumpang naik:

- tambahPenumpang()

Penumpang turun:

- hapusPenumpang()

tambahPenumpang(namaPenumpang, arrayPenumpang)

- Jika angkot kosong, penumpang naik dan duduk di kursi pertama
- Penumpang berikutnya duduk di kursi selanjutnya secara berurutan
- Jika ada kursi karena penumpang turun, penumpang yang naik berikutnya duduk di kursi kosong terlebih dahulu
- Asumsi kursi tidak akan penuh dan akan bertambah terus jika ada penumpang naik
- Nama penumpang tidak boleh sama, untuk menghindari kebingungan ketika nanti penumpang turun

SILAHKAN DIKERJAKAN

JavaScript Dasar 2

```
function tambahPenumpang(namaPenumpang, arrayPenumpang) {
 if (arrayPenumpang.length == 0) {
  arrayPenumpang.push(namaPenumpang)
  return arrayPenumpang
 } else {
  for (let i = 0; i < arrayPenumpang.length; i++) {
   if (arrayPenumpang[i] == namaPenumpang) {
    console.log(namaPenumpang + ' sudah ada di dalam angkot')
    return arrayPenumpang
   }
  }
  for (let i = 0; i < arrayPenumpang.length; i++) {
   if (arrayPenumpang[i] == undefined) {
    arrayPenumpang[i] = namaPenumpang
    return arrayPenumpang
   }
  }
  arrayPenumpang.push(namaPenumpang)
  return arrayPenumpang
 }
}
```

hapusPenumpang(namaPenumpang, arrayPenumpang)

- Jika angkot kosong, tampilkan pesan bahwa angkot kosong
- Jika ada penumpang yang namanya sesuai, hapus nama penumpang pada array dengan memberi nilai undefined
- Jika tidak ada penumpang yang namanya sesuai, tampilkan pesan kesalahan

```
function hapusPenumpang(namaPenumpang, arrayPenumpang) {
  if (arrayPenumpang.length == 0) {
    console.log('Angkot masih kosong')
    return arrayPenumpang
  } else {
    for (let i = 0; i < arrayPenumpang.length; i++) {
      if (arrayPenumpang[i] == namaPenumpang) {
        arrayPenumpang[i] = undefined
        return arrayPenumpang
      } else if (i == (arrayPenumpang.length - 1)) {
        console.log(namaPenumpang + ' tidak ada di dalam angkot')
        return arrayPenumpang
      }
    }
  }
}
```

```
// array method – includes

let arr = [1, 2, 6, 8]

arr.includes(6) // true
arr.includes(14) // false
```

```
// array method – indexOf

let arr = [1, 2, 6, 8]
// indexOf(searchElement, fromIndex)

arr.indexOf(6) // 2
arr.indexOf(14) // -1
```

REFACTOR TIME

JavaScript Dasar 2

```
function tambahPenumpang(nama, penumpang) {
  if (penumpang.length === 0) {
    penumpang.push(nama);
    return penumpang;
  }
  if (penumpang.includes(nama)) {
    console.log(nama + ` sudah ada di dalam angkot`);
    return penumpang;
  }
  const kosong = penumpang.indexOf(undefined);
  if (kosong !== -1) {
    penumpang[kosong] = nama;
    return penumpang;
  }
  penumpang.push(nama);
  return penumpang;
}
```

```
function hapusPenumpang(nama, penumpang) {
  if (penumpang.length === 0) {
    console.log("Angkot masih kosong");
    return penumpang;
  }
  const index = penumpang.indexOf(nama);
  if (index !== -1) {
    penumpang[index] = undefined;
    return penumpang;
  }
  console.log(nama + ` tidak ada di dalam angkot`);
  return penumpang;
}
```

Object

Array

kumpulan elemen, yang memiliki **index**.

Object

kumpulan elemen, yang memiliki **nama**.

Object

object adalah **array**, yang lebih **sakti**.

“In JavaScript, object are king. If you understand object, you understand JavaScript.”

Object pada bahasa pemrograman lain

- Python: Dictionaries
- PHP: Associative Array
- C: Hash Tables
- Java: Hash Maps
- Ruby & Perl: Hashes

Kenapa object?

```
let namaMhs = “Jainal Arthur Sibuea”
let umurMhs = 41
let lulus = true
let IPSemester = [3.60, 3.10, 3.25, 3.04]

function IPKumulatif(IPSemester) {
  let total = 0
  for (let i = 0; i < IPSemester.length; i++) {
    total += IPSemester[i]
  }
  return total / IPSemester.length
}
```

```
let mhs = [“Jainal Arthur Sibuea”, 41, true, [3.60, 3.10, 3.25, 3.04]]

function IPKumulatif(IPSemester) {
  let total = 0
  for (let i = 0; i < IPSemester.length; i++) {
    total += IPSemester[i]
  }
  return total / IPSemester.length
}

IPKumulatif(mhs[3])
```

```
let mahasiswa = {
  nama: "Jainal Arthur Sibuea",
  IPSemester: [3.60, 3.10, 3.25, 3.04],
  IPKumulatif: function() {
    let total = 0
    let ips = this.IPSemester
    for (let i = 0; i < ips.length; i++) {
      total += ips[i]
    }
    return total / ips.length
  }
}
```

Cara membuat object

Object Literal

```
let mahasiswa = {
  nama: "Jainal Arthur Sibuea",
  umur: 41,
  lulus: true,
  IPSemester: [3.60, 3.10, 3.25, 3.04],
}
```

Function Declaration

```
function buatObjectMahasiswa(nama, nim, email) {
  let mahasiswa = {}
  mahasiswa.nama = nama
  mahasiswa.nim = nim
  mahasiswa.email = email
  return mahasiswa
}

let mhs1 = buatObjectMahasiswa("Jainal", "4342411019", "jainalgay@gmail.com")
```

Constructor

```
function Mahasiswa(nama, nim, email) {
  this.nama = nama
  this.nim = nim
  this.email = email
}

let mhs1 = new Mahasiswa("Jainal", "4342411019", "jainalgay@gmail.com")
```

this

```
// global scope

let a = 10
console.log(window.a)

function halo() {
  console.log(this)
  console.log('halo')
}
this.halo()
```

```
let obj =  {a: 10, b: 15}
obj.halo = function () {
  console.log(this)
}

obj.halo()
```

```
function Halo() {
  console.log('halo')
  console.log(this)
}

new Halo()
let obj1 = new Halo()
let obj2 = new Halo()
```

Latihan latihan latihan!!



Studi Kasus Angkot

Yap, lagi dan lagi, gatau mau nulis apa

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer elementum metus nec viverra suscipit. In feugiat magna sapien, varius pellentesque dolor fringilla non.

SpankCode

Thankyou

Spank your bugs away



What We Offer

Tailored Solutions for the Digital Age

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer elementum metus nec viverra suscipit. In feugiat magna sapien, varius pellentesque dolor fringilla non. Mauris vitae lacinia leo, vitae eleifend lorem. Quisque facilisis nulla erat, at commodo lacus ultrices nec. Nam at orci in augue auctor rutrum sed id massa.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer elementum metus nec viverra suscipit. In feugiat magna sapien, varius pellentesque dolor fringilla non. Mauris vitae lacinia leo, vitae eleifend lorem. Quisque facilisis nulla erat, at commodo lacus ultrices nec. Nam at orci in augue auctor rutrum sed id massa.